

Toc H Institute of Science & Technology
(An ISO 9001:2015 Certified Institution)
Arakkunnam

Date: 25/08/25



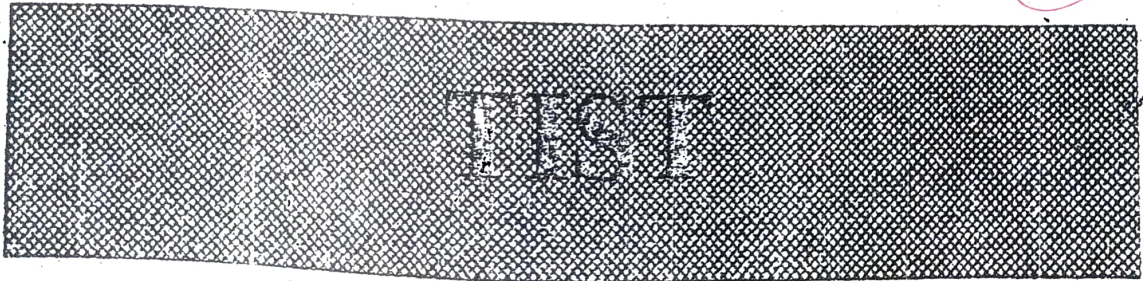
INTERNAL ASSESSMENT MAIN ANSWER BOOKLET

	I	II	III	IV	V	VI	VII	VIII	IX	X	XI	XII	XIII
a	3	3	—	3	—	6	3	2					
b						7	7						
c													
d													
e													
f													
g													
h													
Total													
Grand Total													

TOTAL

34/50

34



Name	Karyap P												
Branch & Semester	CSE SS IB												
Subject with Code	CS 305 SYSTEM SOFTWARE												
Roll No.	14	University Reg. No.	T	0	C	2	3	C	S	0	7	8	
Signature of Invigilator							No. of Additional Sheets						

1) System Software

- These are system dependent
- They can run independently and provide platform for application software

- ~~These~~ System Software does not instruct users or uses as they work in the background
eg: linker, loader

Application Software

- These are system independent
- They cannot run independently can only perform in the presence of system software.
- ~~These~~ User Interaction takes place in application software
eg: Microsoft office package.

2) ~~START~~: used to name and find the starting address of a program.

2) ~~START~~: used to specify the name and starting address of a program

END: used to specify the end of a program

RES W: used to reserve no. of words in a program.

RES B: used to reserve no. of bytes in a program.

6) Linker

- Linking is the process of collecting and combining small codes and combining them into a single file and is loaded to the memory for execution.

- Instead of compiling an entire application as single unit it can be decomposed into small units and individual compilation can take place.

- Linking can be done while runtime and compilation ~~time~~ time.

Loader

- Loader is a utility of Operating System
- Loader is used to copy a program from the storage device and to the main memory so that execution can take place.
- Definition, invocation, expansion.
- It calculates the size of the program and provides space in the main memory.

Macroprocessor

- This program copies a stream of text from one place to another.
- macro are a special code segment that are only defined once in a program and are repeated by called upon when its usage comes up.
- This allows the user to write a short hand program.

Text Editor

- Text Editor is a program used to edit plain text.
- The Text Editor allows the user to write a series of no sum of text in the main memory.
- The editing, uploading, deleting all these process can be done using Text Editor.
- The only drawback is that ~~that~~ these features are only limited for fonts.

6) b) The SIC / nE machine architecture can be represented w.r to respect to memory, registers, data format, instruction format, instruction set, input address, modes Input and output.

Memory:

- The SIC / nE has a similar Memory definition as SIC
- The minimum memory space for SIC / nE is 2^{40} bytes
- Three consecutive bytes form a word.
- All the addresses are in one byte format.

Registers

- B Base register
- S general purpose register
- T general purpose register.
- F Floating point accumulator.

In addition to these registers registers in the SIC are also used.

- A Accumulator register For automatic operations ADD, SUB
- X Index register For addressing.
- L Linkage register For linkage process.
- PC Program Counter
- SW Status word.

Data Format

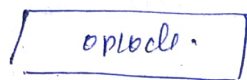
The SIC / nE has a similar data format as a SIC

- a 48 bit floating point is used in this architecture.

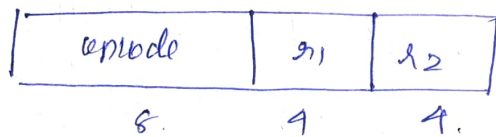
- characters are represented by 8-bit ASCII codes
- are represented by 24 bit binary numbers

Instruction Format.

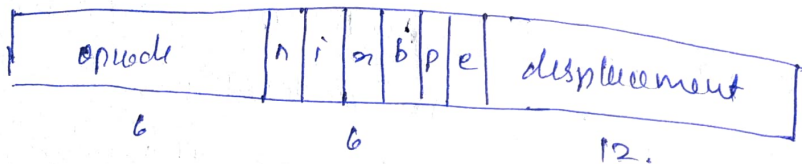
Format 1: only ^{6 bit} opcodes are used in this format



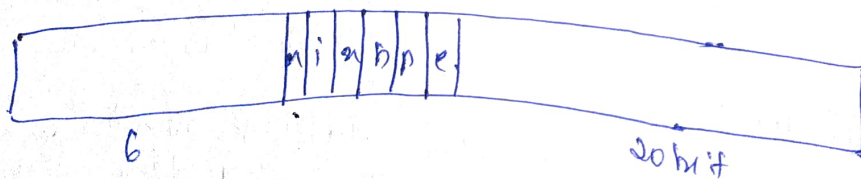
Format 2: 8 bit opcoding code and 2 registers with 4 bits are used in this format r_1 and r_2



Format 3: 8 bit opcoding code and 6 flags 6 bits r_1 6 flags and 12 bits for the displacement of the address at the operand.



Format 4: 6 bit of opcoding code and 6 bits r_1 6 flags and 20 bit for the address.



Instruction Set

All the instruction set in the 8086 are also defined in 8088/8086.

Data Movement Instruction: In this instruction storing and loading takes place.

D
A
C
S
S.

Arithmetic Instruction: In this instruction arithmetic operations takes place. In addition to it special arithmetic operation on registers takes place here.

ADD, SUB, DIV, MUL, ADDP, SUBP, MULP.

Compare Instruction: In this instruction data is compared and results are modified in $>, <, =$

In 8088/8086, instructions for floating point takes place.

ADDP, SUBP, DIVP.

Jump Instruction: Jump instruction are used to jump from one line to another line.

Subroutine Instruction: where RESB Reserve bytes and RESW Reserve word operation takes place.

Addressing Modes

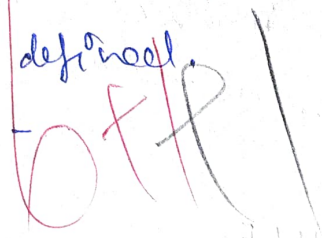
- 1) Base Relative
- 2) Program Relative.
- 3) Base Relative Indexing
- 4) Program Relative addressing.
- 5) Direct Modes.

6) Indexed Mode.

7) Immediate Mode.

Input and Output

- In this Input and output of SIC/XE are defined.



b) Begin

Read the First Input Line.

IF opcode = 'START' Then

Begin.

Save # [opcode] as starting address

Initialize LOCCTR to starting address.

write line to the Intermediate file.

read the next Input Line.

and else

Initialize LOCCTR to 0.

and {if start}

while OPCODE $\neq$ end do.

Begin

IF this is not a comment line then

Begin

if there is a symbol in the label then.

Begin

search label in symbol table

if found then

~~nothing~~

set an error flag (duplicate symbol)

else

insert (label, locctr) to symbol table

end { if symbol }

~~Search opcode~~ ~~opcode~~

Search OPDAB the opcode

add 8 (length of instruction) to locctr.

else if OPDAB = 'WORD' then

add 8 to locctr

else if OPDAB = 'RESW' then

add 8 * # [OPERAND] to locctr.

else if OPDAB = 'RESB' then

add # [OPERAND] to locctr.

else if OPDAB = 'BYTE' then

begin

find the length of the constant in bytes

add length to locctr.

end { if BYTE }

else

set error flag (invalid opcode)

end { if not comment line }

write this line to the intermediate file
read the next input line

end { if end }

Write this to my Index medical file.

Start { LocCTR = starting address }

End { Pass 13 }.

Q

Location	Label	OpCode	Operand	
4000	TEST	START	4000	4000
4000	FIRST	LDA	FIVE	4003
4003		STA	ALPHA	4006
4006	ALPHA	RESW	2	400C
400C	FIVE	WORD	5	4000
4000				
4000	LDA			
4003	STA			
4006	RESW			
400C	WORD.			

A)

LDA ALPHA.

STA GAMMA.

LDA BETA.

STA ~~BETA~~.ALPHA.

LDA GAMMA.

STA BETA.

RESW ALPHA 1

RESW BETA. 1

RESW GAMMA. 1

2a)

~~GDP~~

LDA BETA.

MUL HA, BETA.

SUB #9.

STA ALPHA.

RESW BETA

RESW ALPHA.